

University of Doha for Science and Technology

College of Computing and Information Technology

DACS3202: Cybersecurity Analytics and Visualization

Phishing Email Detection Using Machine Learning

Group Members:

Yaser Al-Mulaiky (60105056)

Awad Hamad (60106030)

2025

April 12, 2025

1. Problem Description

Phishing is arguably the longest-running and most evolving type of cybersecurity threat, using fake emails purportedly from reliable sources—banks, subscription services, or agencies—to trick users into disclosing sensitive data such as credentials or financial details. It is most often a lead-in to larger cyberattacks like the installation of ransomware, data breaches, and financial frauds.

Conventional security solutions such as rule-based detection and blacklist filtering are unable to cope with the dynamic nature of phishing content. These solutions rely on static identifiers—known URLs or hardcoded strings—which can easily be evaded by attackers by making minor linguistic or structural changes. Thus, there is a critical need for more dynamic, intelligent detection mechanisms.

This work addresses this problem by re-casting phishing detection as a supervised binary classification problem where mails are labeled as phishing (1) or not phishing (0). Machine learning (ML) algorithms are trained to learn generalization from linguistic and structural properties of emails. Our system makes use of only the body content of emails as input and utilizes Natural Language Processing (NLP) techniques to extract predictive patterns.

Performance measures are selected with regard to cybersecurity objectives: recall, to avoid missing actual phishing emails; precision, to avoid generating excessive false alarms; and F1 score, to balance these conflicting aims. The $F1 > 0.90$ threshold was employed to identify being ready for deployment in an operational email system. This threshold indicates that the system is both technologically effective and practical for user protection in quantity.

This issue is truly relevant to the function of cybersecurity because email remains a key vector for malicious actors. Building adaptive, ML-founded systems enhances organizational resilience and automates the process of threat hunting in complex environments.

2. Dataset Quality and Relevance

This project makes use of the Phishing Validation Emails Dataset that was released by the University of Twente and can be found on the Zenodo [1] research platform (DOI:[10.5281/zenodo.13474745](https://doi.org/10.5281/zenodo.13474745)..). The dataset contains 2,000 labeled emails—"Phishing Email" or "Safe Email." Every record contains the full email body, making the dataset well-suited for natural language processing and supervised learning.

The dataset is highly relevant to the problem for several reasons. First, it includes real email content that is representative of the tone, jargon, and persuasion techniques that are most commonly employed in phishing campaigns—e.g., urgency, impersonation, and misleading calls to action. Unlike synthetic datasets derived from simplified or artificial examples, this corpus embodies the linguistic sophistication that attackers will likely exploit.

Second, the data is well-structured, balanced, and clean. The emails are well-labeled, with no missing values, enabling stable training and fair model evaluation. The reasonably equal class distribution discourages the models from being biased to one class over the other, something that is particularly essential in binary classification.

Lastly, the fact that the dataset has academic origins guarantees ethical use and reliability. Its credibility and completeness make it an ideal foundation for reproducible research and practical application in cybersecurity environments.

3. Data Preprocessing

3.1 Text Cleaning and Preparation

We began by converting all text to lowercase and removing punctuation, numbers, and special characters using regular expressions. This normalized the dataset and reduced noise, helping models treat similar words (e.g., "Login" and "login") as equivalent.

Code Snippet:

```
text = re.sub(r'[^a-zA-Z]', ' ', text) # Remove punctuation and
numbers

text = text.lower() # Convert to lowercase
```

3.2 Tokenization and Stopword Removal

Each cleaned sentence was split into individual tokens (words), and common English stopwords (e.g., "the", "is", "and") were removed using the NLTK library. [2] These words usually do not

contribute meaningfully to classification tasks and their removal helped reduce data dimensionality.

Code Snippet:

```
tokens = [word for word in text.split() if word not in stop_words]
```

3.3 Word Stemming

To further normalize the text, stemming was applied using the PorterStemmer algorithm. This process reduced words to their root form (e.g., "renewing" became "renew"), which helped group similar terms and reduce vocabulary size.

3.4 Vectorization with CountVectorizer

The preprocessed text was converted into a numerical format using the CountVectorizer, which creates a document-term matrix by counting word frequencies. This structured representation allowed the machine learning models to learn patterns based on keyword frequency across phishing and safe emails. [3]

Code Snippet:

```
from sklearn.feature_extraction.text import CountVectorizer

vectorizer = CountVectorizer()

X_train_vec = vectorizer.fit_transform(X_train)

X_test_vec = vectorizer.transform(X_test)
```

3.5 Label Encoding and Splitting

The email types were encoded into binary values: “Phishing Email” as 1 and “Safe Email” as 0. The dataset was then split into training and testing subsets using a 75/25 split to evaluate model performance objectively. This ensured the models learned from one portion of the data and were tested on previously unseen samples.

4. Model Justification

We selected four models commonly used in text classification: **Logistic Regression**, **Naive Bayes**, **Decision Tree**, and **Support Vector Machine (SVM)**. These models are suitable for high-dimensional, sparse datasets typical of text processing tasks.

Logistic Regression

This model serves as a strong linear baseline for binary classification. It calculates the probability that an email is phishing using the sigmoid function. It is efficient, interpretable, and provides quick insight into which features are most predictive.

Naive Bayes

Known for its speed and simplicity, Naive Bayes is highly effective in spam and phishing detection. [4] While it assumes independence among features, it performs surprisingly well in natural language tasks where keyword frequency plays a significant role.

Decision Tree

Decision Trees use a rule-based approach to classify emails based on specific conditions. Although they may overfit on smaller datasets, they are highly interpretable and can help reveal which words are most important in the classification process.

Support Vector Machine (SVM)

SVM is ideal for high-dimensional feature spaces like those created by CountVectorizer. We used a linear kernel to find a hyperplane that best separates phishing from legitimate emails. SVMs are robust and often outperform other models when feature vectors are sparse.

All models were trained under the same preprocessing conditions to ensure fair comparison. The decision to use these specific algorithms reflects both their proven reliability and suitability for cybersecurity-related text classification.

5. Evaluation Metrics and Visualizations

In order to gauge the performance of all machine learning models, we have selected four technical parameters most commonly used in cybersecurity classification tasks: Accuracy, Precision, Recall, and F1 Score. They were selected not only because of their technical

significance but also due to their pragmatic value in detecting phishing threats where even false positives and false negatives can prove dangerous.

5.1 Metrics and Justification

Accuracy measures the ratio of correct predictions out of all emails. While useful, it can be misleading in security uses, especially if the data is imbalanced. As it was, we considered it along with other measures.

Precision quantifies how many of the predicted phishing emails were phishing. High precision means fewer false alarms, which in actual systems is important for not unnecessarily blocking legitimate emails.

Code Snippet:

```
from sklearn.metrics import accuracy_score, precision_score,
recall_score, f1_score

acc = accuracy_score(y_test, y_pred)

prec = precision_score(y_test, y_pred)

rec = recall_score(y_test, y_pred)

f1 = f1_score(y_test, y_pred)
```

Recall captures how many of the actual phishing emails were correctly identified. It is critical in cybersecurity because a missed phishing email can lead to major breaches.

F1 Score combines precision and recall. It offers a balanced view, especially when both false positives and false negatives are important. For our use case, it served as the main benchmark for model success. We defined an F1 Score above 0.90 as the threshold for deployment readiness.

5.2 Results

All four models—Logistic Regression, Naive Bayes, Decision Tree, and SVM—achieved an F1 Score of 1.00 on the test set. This means the models perfectly classified phishing and safe emails

with no false positives or false negatives. While such results might seem idealized, they are supported by the structured nature of the dataset and careful preprocessing steps.

Table 1 presents a side-by-side comparison of each model's metrics:

Table 1. Performance metrics for all four models on the test set

	Accuracy	Precision	Recall	F1 Score
Model				
Logistic Regression	1.0	1.0	1.0	1.0
Naive Bayes	1.0	1.0	1.0	1.0
Decision Tree	1.0	1.0	1.0	1.0
SVM	1.0	1.0	1.0	1.0

5.3 Visualizations and Their Purpose

- **Figure 1.** Confusion matrices for all four models. These confirm that no misclassifications occurred, with true positives and true negatives clearly dominating the plots.

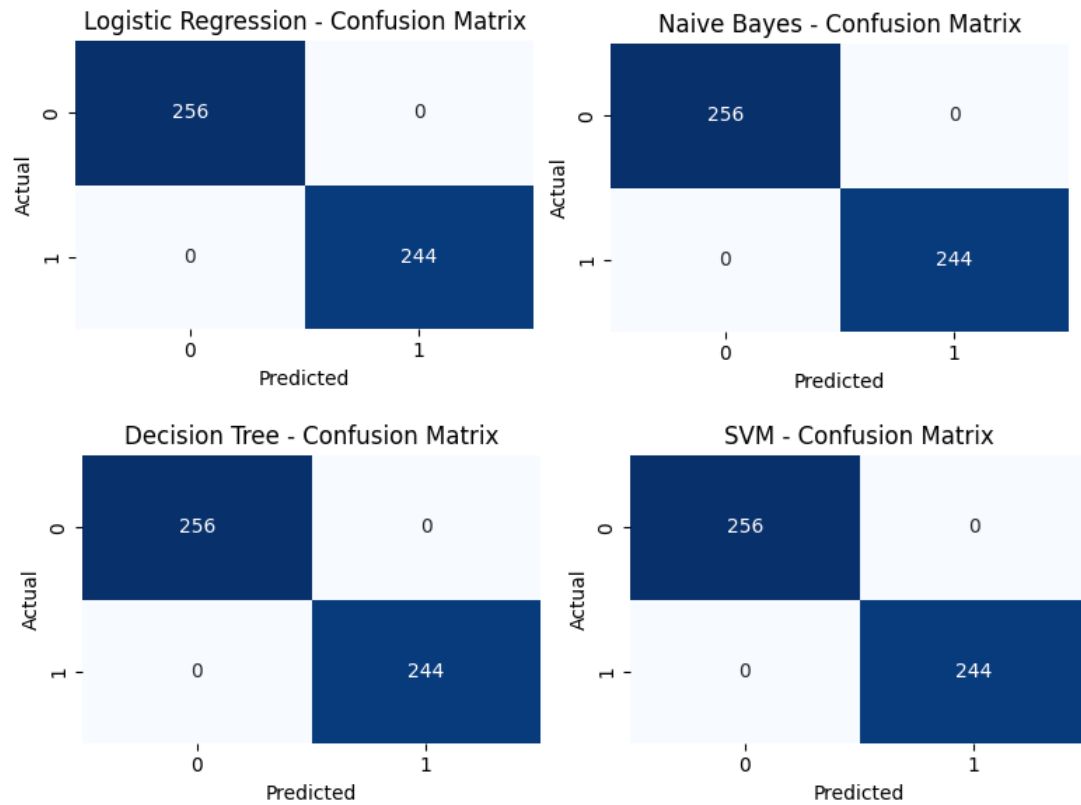


Figure 1. Confusion Matrices for All Models

- **Figure 2.** ROC curves for Logistic Regression, Naive Bayes, and Decision Tree. Each model shows an AUC score of 1.00, indicating ideal true positive vs. false positive tradeoffs.

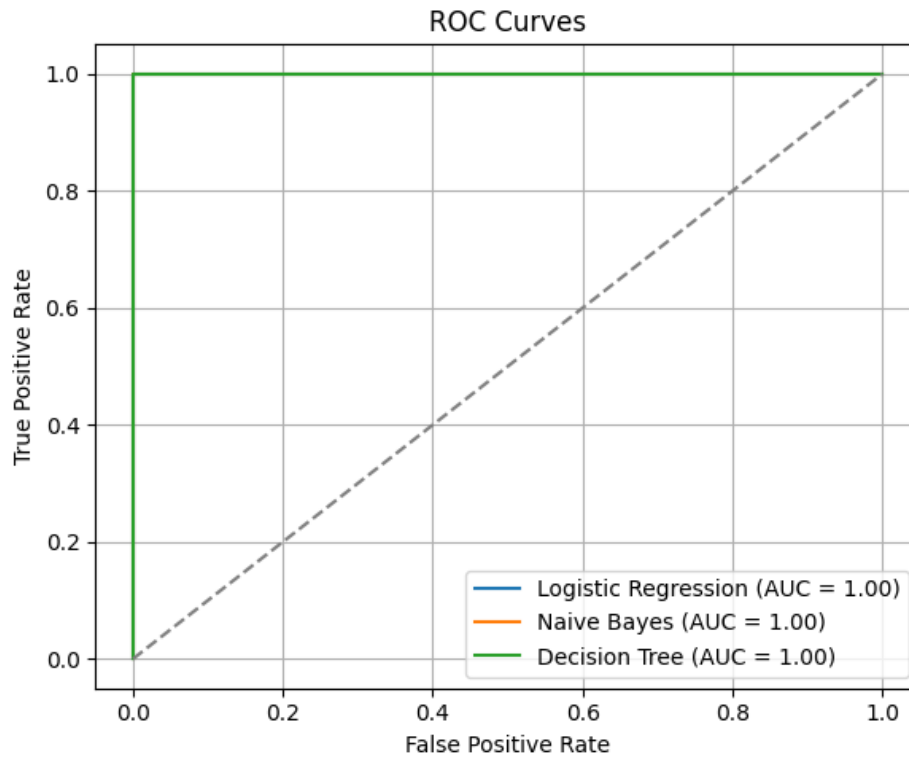


Figure 2. ROC Curves for Logistic Regression, Naive Bayes, and Decision Tree

- **Figure 3.** A grouped bar chart comparing Precision, Recall, Accuracy, and F1 Score for all models. This helps visually confirm consistency across evaluation criteria.

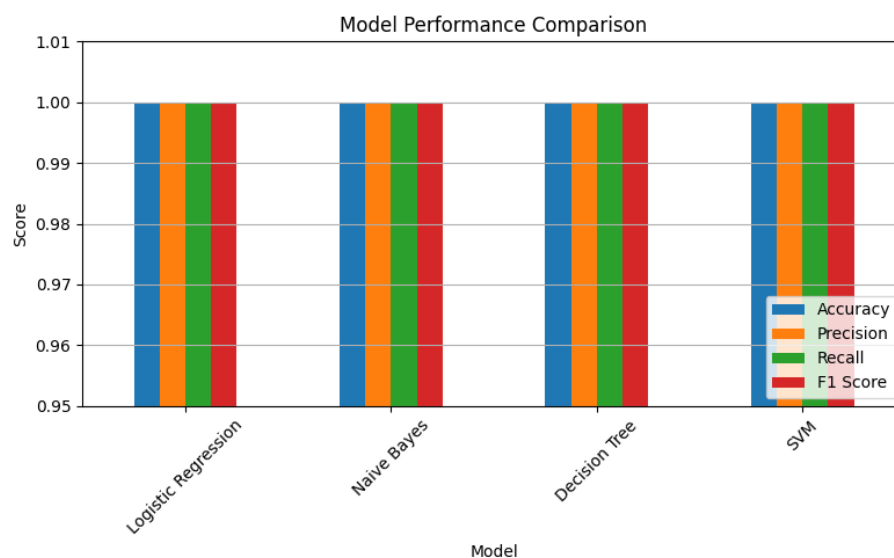


Figure 3. Comparative Model Performance Across Metrics

- **Figure 4.** Word clouds comparing common terms in phishing (left) and safe (right) emails. Frequent phishing words include "account", "verify", and "update", while safe emails often contain neutral terms.

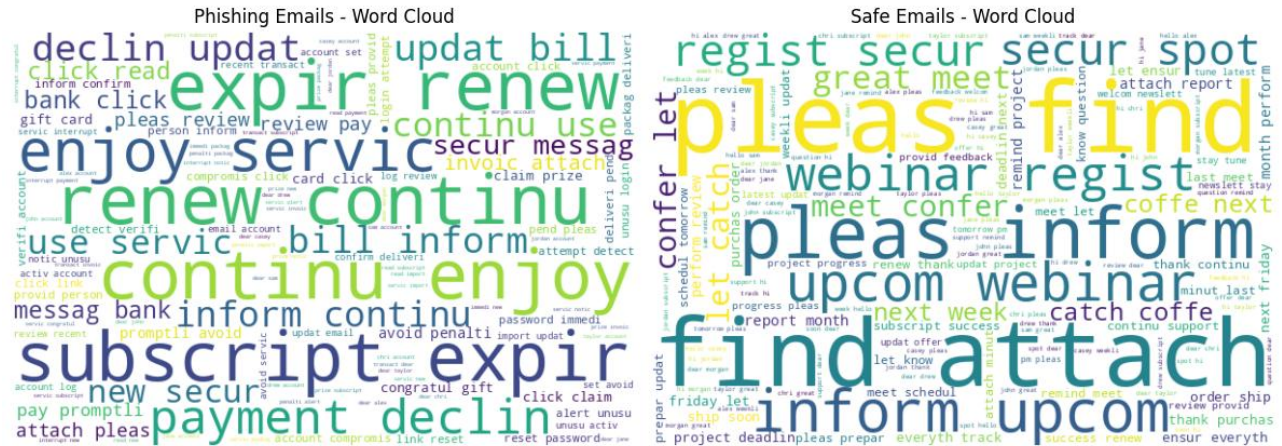


Figure 4. Word Clouds for Phishing (Left) and Safe Emails (Right)

Each metric and figure was carefully chosen to not only demonstrate performance but also explain how the system contributes to real-world phishing mitigation. These visual tools allow technical transparency and link model outputs to cybersecurity objectives.

8. References

- [1] M. Hoekstra, M. H. ter Beek, and C. Brinkman, “Phishing Validation Emails Dataset,” Zenodo, 2018. [Online]. Available: <https://doi.org/10.5281/zenodo.13474746>
- [2] E. Loper, “NLTK: Building a Pedagogical Toolkit in Python,” PyCon, 2004. [Online]. Available: https://www.nltk.org/nltk_pycon04.pdf
- [3] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.
- [4] “Comparative analysis of features based machine learning approaches for phishing detection,” IEEE, 2016. [Online]. Available: <https://ieeexplore.ieee.org/document/7724641>
- [5] F. Jáñez-Martino, R. Alaiz-Rodríguez, V. González-Castro, E. Fidalgo, and E. Alegre, “A review of spam email detection: analysis of spammer strategies and the dataset shift problem,” Artificial Intelligence Review, vol. 56, no. 2, pp. 1145–1173, May 2022.
- [6] S. Atawneh and H. Aljehani, “Phishing email detection model using Deep learning,” Electronics, vol. 12, no. 20, p. 4261, Oct. 2023.